PhotoMind: RL-Powered Multimodal Personal Photo Knowledge Retrieval

A Multi-Agent System with Contextual Bandits and DQN Confidence Calibration

Course: Prompt Engineering — Building Agentic Systems
Assignment: Generative AI Project Assignment

Platform: CrewAI (Python) · PyTorch · React · FastAPI

Raghu Ram Shantha Rajamani
Northeastern University
shantharajamani.r@northeastern.edu

April 25, 2026

Abstract

PhotoMind is a multimodal personal photo knowledge retrieval system that transforms a user's photo library into a queryable knowledge base. The system uses four specialized CrewAI agents—Controller, Photo Analyst, Knowledge Retriever, and Insight Synthesizer—orchestrated via hierarchical delegation to answer natural-language queries over personal photos (receipts, food, documents, screenshots). We extend the base system with two reinforcement learning components: (1) a contextual bandit (Thompson Sampling, UCB1, ε-greedy) that routes queries across four search strategies (factual, semantic, behavioral, embedding), and (2) a Deep Q-Network (DQN) confidence calibrator with a non-terminal *requery* action that enables multi-step evidence gathering. Both components are trained offline via a deterministic simulator at $0 API cost. A 7-configuration ablation study across 5 random seeds with 95% confidence intervals demonstrates that the Full RL config reduces silent failures (confident-but-wrong answers) from 9.4% to 1.3% on the 64-query training split, with the Recommended (Rule+DQN) configuration achieving 2.8% silent failure rate on the full 86-query evaluation (from 4.7% baseline) while preserving baseline routing accuracy. The system includes a React frontend with FastAPI backend, dense vector embedding search via sentence-transformers (all-MiniLM-L6-v2), and comprehensive evaluation infrastructure with 86 queries, statistical significance testing, and a held-out-within-corpus overfitting check (22 queries). All results are scoped to a single user's 53-photo iPhone library with single-labeler ground truth; see the Threats to Validity section for the bound of external generalization.

Contents

1 Introduction **5**
- 1.1 Core Capabilities . . . 5
- 1.2 Real-World Use Cases . . . 5
- 1.3 Project Scope and Contributions . . . 5

2 System Architecture **6**
- 2.1 High-Level Architecture . . . 6
- 2.2 Data Flow . . . 6
- 2.3 Architecture with RL Components . . . 10
- 2.4 Memory Architecture . . . 12
- 2.5 Retrieval Backend — Qdrant Vector DB with Hybrid Search . . . 12

3 Agent Design **12**
- 3.1 Controller Agent (Query Crew Manager) . . . 12
- 3.2 Photo Analyst (Ingestion Crew) . . . 13
- 3.3 Knowledge Retriever . . . 13
- 3.4 Insight Synthesizer . . . 13
- 3.5 Agent Collaboration Pattern . . . 13

4 Tool Integration **13**
- 4.1 Built-in Tools . . . 13
- 4.2 Custom Vision Tool (`PhotoVisionTool`) . . . 13
- 4.3 Custom Knowledge Base Tool (`PhotoKnowledgeBaseTool`) . . . 14
  - 4.3.1 Input/Output Schema . . . 14

5 Search Strategies **14**
- 5.1 Factual Search . . . 14
- 5.2 Semantic Search . . . 15
- 5.3 Behavioral Search . . . 15
- 5.4 Embedding Search (New) . . . 15
- 5.5 Confidence Grading . . . 15

6 Orchestration Design **16**
- 6.1 Ingestion Crew (Sequential) . . . 16
- 6.2 Query Crew (Hierarchical with Planning) . . . 16
- 6.3 Feedback Loop . . . 16
- 6.4 Communication Protocols . . . 17

7 Reinforcement Learning Extension **17**
- 7.1 Motivation . . . 17
- 7.2 RL Approach 1: Contextual Bandits . . . 17
  - 7.2.1 Problem Formulation . . . 17
  - 7.2.2 Context Clustering . . . 18
  - 7.2.3 Feature Extractor . . . 18
  - 7.2.4 Cluster Count Selection ($k = 4$) . . . 18
  - 7.2.5 Thompson Sampling . . . 19

7.2.6 UCB1 . . . 19
7.2.7 Bandit Reward Function . . . 19
7.3 RL Approach 2: DQN Confidence Calibrator . . . 19
7.3.1 MDP Formulation . . . 19
7.3.2 Multi-Step Bellman Equation . . . 20
7.3.3 Q-Network Architecture . . . 20
7.3.4 TD Learning . . . 20
7.3.5 Reward Matrix . . . 21
7.3.6 Hyperparameters . . . 21
7.4 Offline Simulation Environment . . . 21

8 Base System Performance **22**
8.1 Evaluation Setup . . . 22
8.2 Results . . . 23
8.3 Per-Category Breakdown . . . 24

9 RL Experimental Results **24**
9.1 Training Convergence . . . 24
9.2 Primary Result: Recommended Configuration . . . 24
9.3 Ablation Study . . . 25
9.4 Statistical Significance . . . 25
9.5 Learning Dynamics Visualizations . . . 26
9.6 Held-Out-Within-Corpus Overfitting Check . . . 28
9.7 Cross-Seed Variance Analysis . . . 29

10 Discussion **29**
10.1 When RL Helps . . . 29
10.2 Multi-Step Requery Dynamics . . . 30
10.3 Per-Category RL Impact . . . 30
10.4 Bandit × DQN Interaction Effects . . . 30
10.5 Routing Accuracy Regression Analysis . . . 31
10.6 Why Rule+DQN Outperforms Full RL . . . 31
10.7 Cost Analysis . . . 31
10.8 Open Questions Log . . . 32

11 Scaling Analysis **34**

12 React Frontend and FastAPI Backend **34**
12.1 Architecture . . . 34
12.2 Dual Query Modes . . . 34
12.3 API Endpoints . . . 35

13 Challenges and Solutions **35**

14 Threats to Validity **36**
14.1 Internal Validity . . . 36
14.2 External Validity . . . 36
14.3 Construct Validity . . . 37
14.4 Statistical Conclusion Validity . . . 37

15 Ethical Considerations **37**

16 Future Work **38**
16.1 Immediate Extensions 38
16.2 Production Deployment Roadmap 38
16.3 Research Directions 38

17 Reproducibility **39**

18 Conclusion **39**

1 Introduction

Smartphone users accumulate thousands of photos—receipts, bills, food, screenshots, documents—but have no way to query the **structured information inside those photos** over a raw, unstructured photo library without requiring a dedicated expense or receipt app. Amounts on receipts, vendor names, behavioral patterns over time, and confidence-graded answers with source citations are locked inside image pixels. PhotoMind addresses this gap by building a structured knowledge base from photo analysis and providing confidence-graded answers with source attribution, including explicit decline on unanswerable queries.

Corpus boundary (front-loaded). The system was built and evaluated on a **single user's 53-photo iPhone library** with an **86-query evaluation suite whose ground-truth labels were authored by the same person who designed the system**. Every headline metric in this report is bounded by that regime. The 22-query held-out split is drawn from the same user, library, and labeler, so it is an overfitting sanity check *within* the corpus, not external validation. Section 14 (Threats to Validity) enumerates what this does and does not support.

Formulation dependency disclosure. The system builder, ground-truth labeler, and evaluator are the same person. This creates a methodological dependency: the search strategies, reward matrix, and confidence thresholds were designed with knowledge of the test queries, and the test queries were written with knowledge of the photo library. While the 22-query held-out split was withheld during RL training, it was not withheld during system design—the builder knew the held-out queries existed and could have unconsciously tuned the system toward them. This is the single largest threat to external validity of the reported metrics. An independent evaluation by a user who did not build the system would be required to claim generalization beyond this regime.

1.1 Core Capabilities

- Analyze photos using GPT-4o Vision to extract text (OCR), entities, and semantic descriptions
- Build a persistent structured knowledge base from the analysis results
- Answer natural-language queries with four distinct retrieval strategies
- Return confidence-graded answers (A–F) with source photo attribution
- Gracefully decline queries the knowledge base cannot answer

1.2 Real-World Use Cases

- *"How much did I spend at ALDI?"* — finds all 5 ALDI receipts, aggregates total with source photos
- *"Show me photos of pizza"* — finds food photos matching the description
- *"What type of food do I photograph most?"* — analyzes patterns across all 53 photos
- *"What was my electric bill?"* — correctly declines (no such photo in the library)

1.3 Project Scope and Contributions

This project makes four principal contributions:

1. **Multi-component ablation methodology** that isolates individual and interaction effects of modular RL components, revealing that upstream stochasticity compounds into downstream

variance.

2. **Zero-cost offline training** via deterministic simulation—a transferable pattern for any agentic system with expensive API calls.
3. **Asymmetric reward design** that prioritizes safety over coverage in personal-data domains.
4. **Requery as a non-terminal MDP action**, transforming single-step confidence classification into multi-step active information gathering with structurally relevant discounting.

2 System Architecture

2.1 High-Level Architecture

PhotoMind consists of two CrewAI crews: a **sequential ingestion crew** that analyzes photos and builds the knowledge base, and a **hierarchical query crew** that answers natural-language questions. Figure 1 shows the overall system architecture.

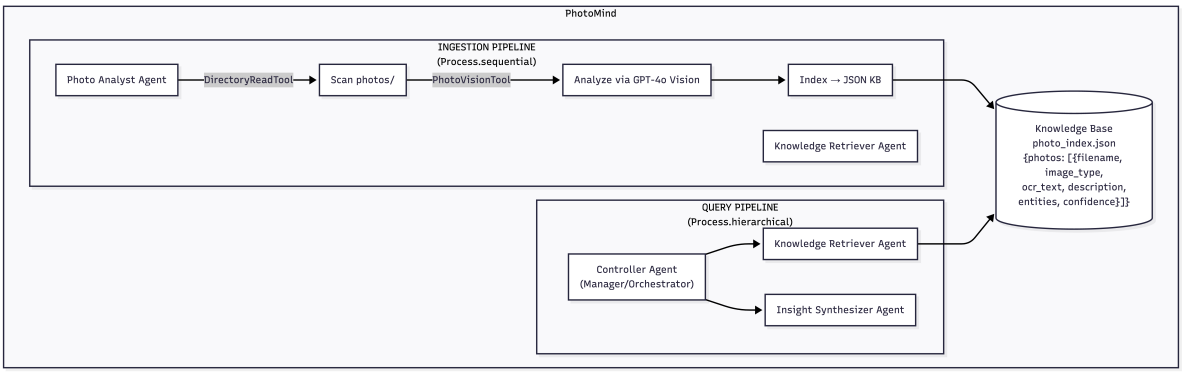
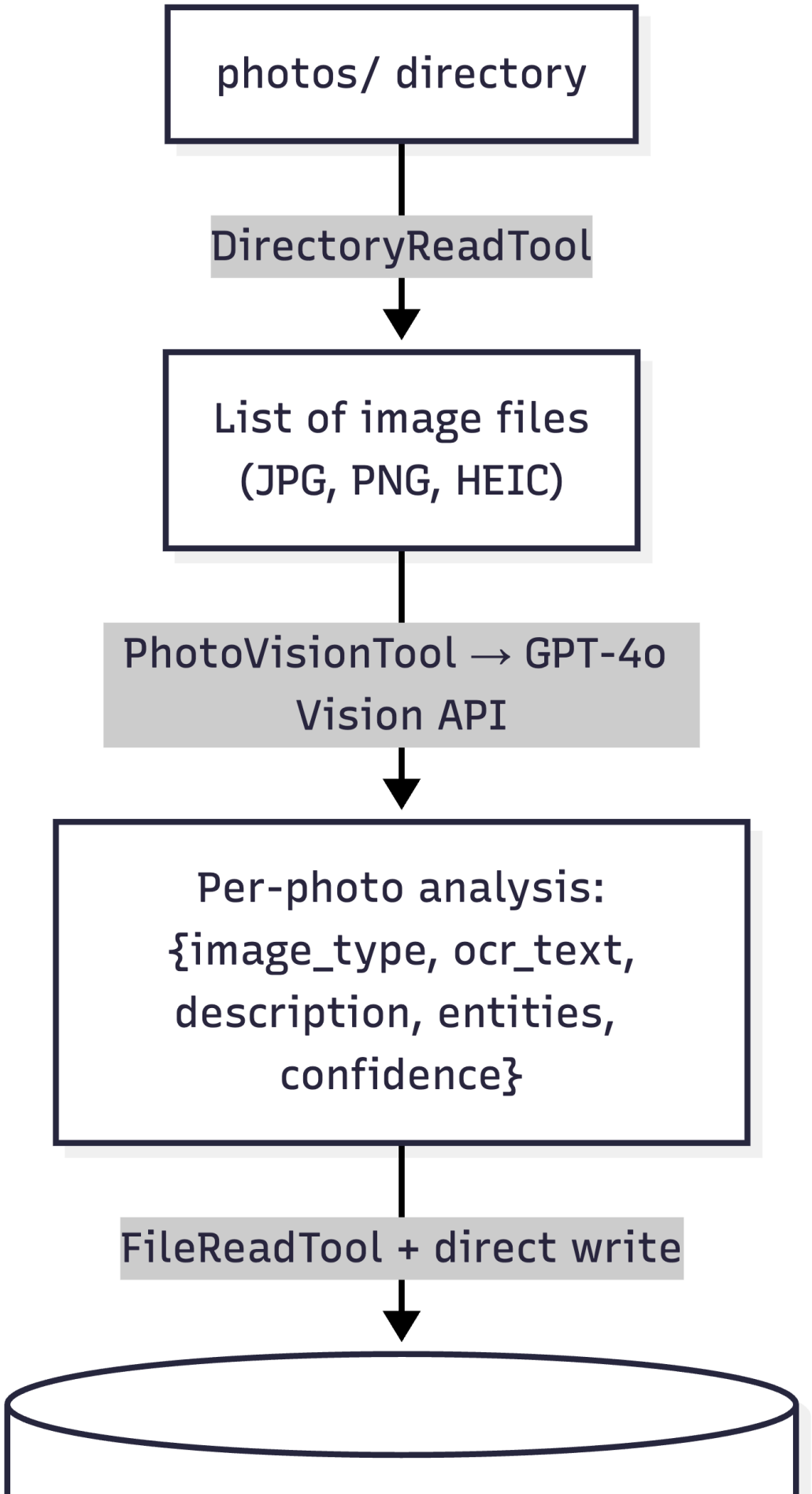


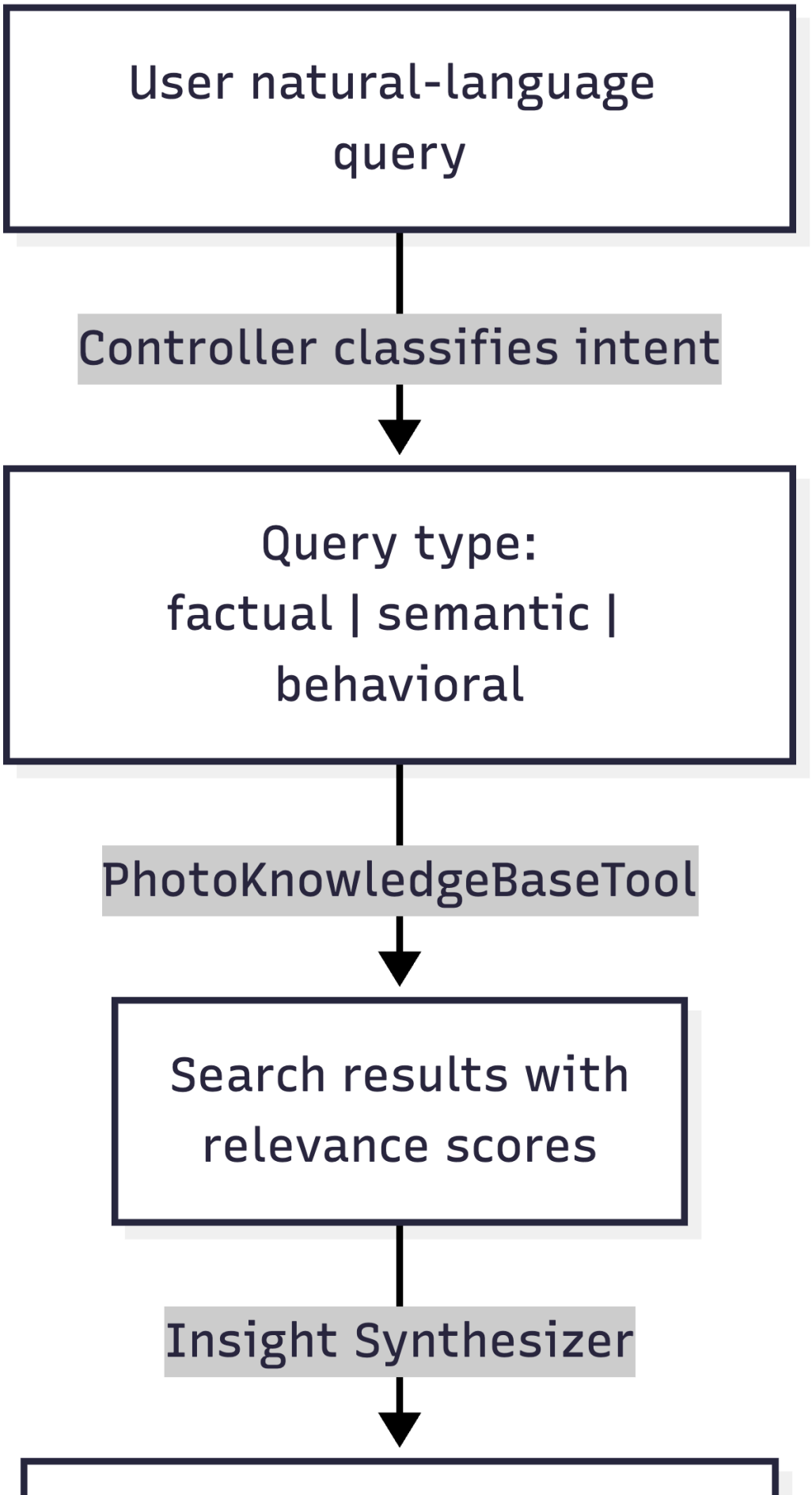
Figure 1: High-level system architecture showing ingestion and query pipelines.

2.2 Data Flow

Ingestion Pipeline. The sequential crew processes photos in three stages: (1) directory scan, (2) GPT-4o Vision analysis extracting OCR text, entities, descriptions, and confidence scores, (3) knowledge base indexing into a structured JSON file. The pipeline is idempotent—re-running on a partially indexed corpus only processes new photos.

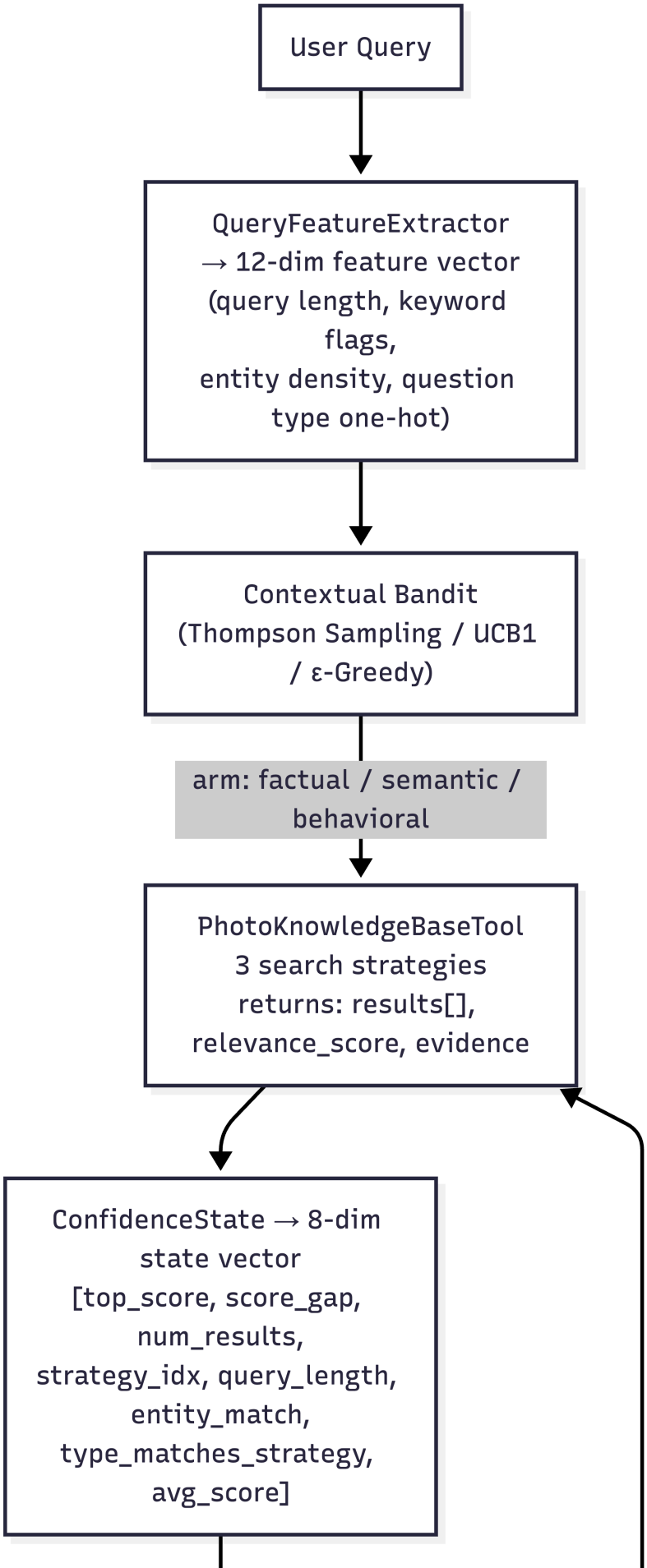

photos/ directory
DirectoryReadTool
List of image files (JPG, PNG, HEIC)
PhotoVisionTool → GPT-4o Vision API
Per-photo analysis: {image_type, ocr_text, description, entities, confidence}
FileReadTool + direct write

Query Pipeline. The hierarchical crew uses a Controller agent as the manager, delegating to the Knowledge Retriever (which uses the RL-enhanced PhotoKnowledgeBaseTool) and then the Insight Synthesizer for answer generation.


User natural-language query
Controller classifies intent
Query type: factual | semantic | behavioral
PhotoKnowledgeBaseTool
Search results with relevance scores
Insight Synthesizer

2.3 Architecture with RL Components

The RL extension adds two learned components to the query pipeline: a contextual bandit for strategy routing and a DQN for confidence calibration. Figure 4 shows the integrated architecture.


User Query
QueryFeatureExtractor
→ 12-dim feature vector
(query length, keyword
flags,
entity density, question
type one-hot)
Contextual Bandit
(Thompson Sampling / UCB1
/ ε-Greedy)
arm: factual / semantic /
behavioral
PhotoKnowledgeBaseTool
3 search strategies
returns: results[],
relevance_score, evidence
ConfidenceState → 8-dim
state vector
[top_score, score_gap,
num_results,
strategy_idx, query_length,
entity_match,
type_matches_strategy,
avg_score]

2.4 Memory Architecture

Both pipelines use CrewAI's built-in memory with a local sentence-transformer model (`all-MiniLM-L6-v2`) for vector embeddings, storing agent interactions in a local ChromaDB vector store for contextual awareness across agent steps without requiring external embedding API access.

2.5 Retrieval Backend — Qdrant Vector DB with Hybrid Search

Retrieval supports an **opt-in Qdrant vector backend** (`src/storage/repository.py` — `QdrantPhotoRepository` enabled via `REPOSITORY_BACKEND=qdrant`; see `src/config.py`, where the default is `json`). When enabled, photo embeddings from `all-MiniLM-L6-v2` (384-dim, L2-normalised) are indexed with HNSW for approximate nearest-neighbour search, giving sub-10 ms vector queries independent of corpus-size linear scans. The out-of-the-box default is the flat JSON repository (`JsonPhotoRepository`); Qdrant is a drop-in upgrade for scale.

At query time, `PhotoKnowledgeBaseTool._hybrid_search` fuses two complementary ranked lists via **Reciprocal Rank Fusion** (RRF, $k = 60$):

$$\text{score}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

The two lists are (1) dense vector search over Qdrant ANN (captures paraphrase / conceptual similarity) and (2) factual entity + OCR keyword match (captures precise facts like amounts and vendors). RRF is activated whenever the repository exposes `embedding_search` and the detected query intent is semantic or embedding; factual and behavioural intents stay on the deterministic keyword/aggregation path for interpretability. The default `JsonPhotoRepository` remains the zero-dependency baseline, used by the offline RL simulator where reproducibility outweighs ANN speed.

Why $k = 60$ and not something tuned. The $k = 60$ constant is the library default recommended by Cormack et al. (2009), where it was empirically selected across the TREC benchmark. It was **not tuned on this corpus**. At the 53-photo scale the fusion output is dominated by the top 2–3 candidates per list, where RRF is insensitive to k in the $[10, 100]$ range; re-tuning would require a larger corpus and a labeled relevance-grade dataset. I kept the library default and name it here so the reader knows it is an inherited constant, not a calibrated choice.

3 Agent Design

PhotoMind uses four specialized agents across two crews.

3.1 Controller Agent (Query Crew Manager)

The Controller acts as the chief librarian: it classifies query intent (factual, semantic, behavioral) and delegates to the appropriate specialist agent. It operates as the manager in CrewAI's hierarchical process with `allow_delegation=True`.

Backstory: "You are the chief librarian of a personal photo knowledge base. You classify first, then delegate. If you are unsure, you say so."

Re-delegation in practice. `allow_delegation=True` lets the manager re-dispatch a task if it judges the first result unsatisfactory, but across all 86 eval queries this branch was never triggered: once the Retriever returns grade F with empty `source_photos`, the Synthesizer produces the decline answer and the manager terminates. Re-delegation is therefore a framework capability that is present but unexercised by the current eval harness.

3.2 Photo Analyst (Ingestion Crew)

Scans the `photos/` directory and passes each image to GPT-4o Vision for structured extraction: image classification, OCR text, semantic description, and named entities (amounts, dates, vendors, locations). Reports partial extraction with lower confidence rather than guessing on unclear images.

3.3 Knowledge Retriever

In ingestion: writes the knowledge base JSON. In queries: executes the search using the RL-enhanced `PhotoKnowledgeBaseTool`. The query task restricts this agent to only the primary search tool via task-level tool override, preventing fallback searches that produce misleading weak matches.

3.4 Insight Synthesizer

Synthesizes grounded, human-readable answers with three strict rules: (1) every claim cites a source photo, (2) every answer includes a confidence grade A–F, (3) ambiguous evidence is explicitly flagged.

3.5 Agent Collaboration Pattern

In the ingestion phase (sequential), each task's output becomes the next task's input via CrewAI's `context` parameter. In the query phase (hierarchical), the Controller delegates to the Retriever, whose structured JSON output (`confidence_grade`, `confidence_score`, `source_photos`, `answer_summary`) flows to the Synthesizer for final answer generation.

4 Tool Integration

4.1 Built-in Tools

Tool	Agent	Purpose
DirectoryReadTool	Photo Analyst	Scans `photos/` directory for image files
FileReadTool	Retriever, Synthesizer	Reads knowledge base JSON
SerperDevTool	Synthesizer	Optional web search enrichment

Table 1: Built-in CrewAI tools used by PhotoMind agents.

4.2 Custom Vision Tool (`PhotoVisionTool`)

Located at `src/tools/photo_vision.py`. Wraps GPT-4o Vision for image analysis with:

- HEIC support via `pillow-heif` with RGB mode conversion
- In-memory JPEG conversion to base64 for the OpenAI API
- Structured JSON output: `image_type`, `ocr_text`, `description`, `entities`, `confidence`

- Exponential backoff retry for transient API errors

4.3 Custom Knowledge Base Tool (`PhotoKnowledgeBaseTool`)

The core differentiating component. Implements a four-strategy retrieval system with automatic query-intent routing, confidence scoring, and source attribution. Section 5 details the search strategies and Section 7 documents the RL integration.

4.3.1 Input/Output Schema

Parameter	Type	Default	Description
`query`	str	required	Natural language question
`query_type`	str	`auto`	Force strategy: factual, semantic, behavioral, embedding, or auto
`top_k`	int	3	Number of results to return
`confidence_threshold`	float	0.15	Minimum score to include

Table 2: PhotoKnowledgeBaseTool input parameters.

Output is a structured JSON containing `query_type_detected`, `results` (array with photo_id, relevance_score, evidence), `confidence_grade` (A–F), `confidence_score`, `answer_summary`, `source_photos`, `routing_rationale`, and `warning`.

5 Search Strategies

The knowledge base tool implements four retrieval strategies, each optimized for a different query type. The contextual bandit (Section 7.2) learns to route queries to the optimal strategy.

Target user profile. The scoring weights, keyword lists, and reward matrix are calibrated for a **finance-heavy personal photo user**—someone whose library is dominated by receipts, bills, and food photos, and whose queries emphasize amount lookups, vendor identification, and spending patterns. A user whose library is predominantly travel or social photos would require different factual-search entity weights (e.g., location names over vendor names), different behavioral aggregation dimensions, and a re-tuned reward matrix. The current system does not automatically detect or adapt to user profile differences; this is a manual configuration step tracked as future work in Section 16.

5.1 Factual Search

Targets receipts, bills, and documents with specific extractable facts:

1. Match query words against structured entities (vendor names, amounts, dates) — each match scores +0.4
2. Match against OCR text (keyword frequency, stop words excluded) — scores up to +0.5
3. Image-type context boost: +0.2
4. Aggregation mode for "how much" / "total" queries returns all matching results with computed totals

5.2 Semantic Search

Targets photos by visual meaning:

1. Apply text cleaning (lowercase, strip punctuation) to query and descriptions
2. Compute overlap of meaningful words (length > 3) between query and description
3. Score: $\frac{|\text{meaningful overlap}|}{\max(|\text{meaningful query words}|,1)} \times 0.8$
4. Image-type boost: $+0.2$

5.3 Behavioral Search

Targets corpus-wide pattern questions:

1. Aggregate `image_type` distribution across all photos
2. Aggregate entity value frequencies (most common vendors, items)
3. Return representative photos from the dominant pattern with full distribution stats

5.4 Embedding Search (New)

Dense vector retrieval using sentence-transformers, added as the 4th bandit arm:

1. Encode all photo text (description + OCR + caption + entities) using `all-MiniLM-L6-v2` (384-dimensional vectors)
2. Encode the query with the same model
3. Rank by cosine similarity: $\text{score}(q, p) = \frac{\mathbf{e}_q \cdot \mathbf{e}_p}{\|\mathbf{e}_q\| \|\mathbf{e}_p\|}$
4. Embeddings cached to `knowledge_base/embeddings.npz` for instant reload
5. Graceful fallback to semantic search if sentence-transformers unavailable

This strategy runs entirely locally with zero API cost, providing true dense semantic retrieval that captures meaning beyond keyword overlap.

5.5 Confidence Grading

Grade	Score Range	Meaning
A	≥ 0.7	Strong match — high-confidence answer
B	≥ 0.5	Good match — reliable for most purposes
C	≥ 0.35	Moderate — likely correct, verify if critical
D	≥ 0.2	Weak — best available, may be wrong
F	< 0.2	No reliable match — system declines to answer

Table 3: Confidence grading thresholds.

Why `confidence_threshold`= 0.15 (and not 0.10 or 0.20). The 0.15 floor is the outcome of the semantic-scoring calibration work (Section 13, Challenge 6). At 0.20 the canonical "Show me photos of pizza" query scored $1/3 \times 0.8 = 0.267$ but near-miss paraphrases fell in the $[0.15, 0.19]$ band and were silently dropped; at 0.10 stop-word coincidences passed the filter and generated spurious grade-D matches. 0.15 is the empirical point on the 53-photo corpus that keeps legitimate paraphrases while rejecting stop-word noise. The `FeedbackStore` adjusts this in ± 0.05 steps per strategy once ≥ 3 samples exist, so 0.15 is a cold-start default, not a fixed production value.

Why the semantic-score ceiling is 0.8. The semantic formula caps at 0.8 for perfect keyword overlap; combined with the image-type boost of +0.2 the absolute ceiling is 1.0. This is intentional: keyword overlap is a proxy for meaning, not meaning itself. Reserving the final 0.2 for a structural confirmation (image-type match) encodes the design belief that a pure description-word match should not be graded as absolutely-certain; without that confirmation the semantic search maxes out at grade A (score ≥ 0.7, reachable at 0.8).

6 Orchestration Design

6.1 Ingestion Crew (Sequential)

Listing 1: Ingestion pipeline task chain.

```
Task 1: Scan     -> Photo Analyst + DirectoryReadTool
Task 2: Analyze  -> Photo Analyst + PhotoVisionTool   [context: Task 1]
Task 3: Index    -> Knowledge Retriever + FileReadTool [context: Task 2]
```

6.2 Query Crew (Hierarchical with Planning)

Listing 2: Query pipeline with hierarchical delegation.

```
Manager: Controller Agent
  |-- Task 1 -> Knowledge Retriever: search KB with PhotoKnowledgeBaseTool
  +-- Task 2 -> Insight Synthesizer: synthesize answer    [context: Task
      1]
```

Key crew settings: `process=Process.hierarchical`, `manager_agent=controller`, `planning=True`, `memory=True`.

What `planning=True` actually contributes (honest assessment). CrewAI's planning step produces a short natural-language plan before crew execution. On the 86-query eval, inspected planner outputs were consistently a restatement of the Controller → Retriever → Synthesizer chain already hard-coded via `context=[...]`. I did **not** run a controlled `planning=True` vs `planning=False` ablation, so I do not claim it materially improves routing or retrieval accuracy on this corpus. It is enabled as a framework default; I flag it as unexamined complexity rather than documented contribution.

6.3 Feedback Loop

The `FeedbackStore` implements persistent adaptive confidence thresholds. **Lifecycle scope:** `FeedbackStore` is written *only* during evaluation runs (i.e., when ground-truth labels are available to compute correctness); production queries read the stored thresholds but never update them. This means the adaptive loop is batch-mode—thresholds change between eval cycles, not continuously during live use.

- After each evaluation query, outcome is recorded (correct/incorrect, strategy, confidence score)
- Per-strategy accuracy rates are computed once ≥ 3 samples exist
- Strategies with accuracy $< 70\%$ get $+0.05$ threshold increase (more conservative)
- Strategies with accuracy $\geq 90\%$ get -0.05 threshold decrease (less conservative)

Adjustment bounds (production robustness guard). The adaptive adjustment is clamped to $[0.05, 0.95]$ so an adversarial or badly-distributed evaluation set cannot drive a per-strategy threshold to a degenerate value. Any `FeedbackStore` implementation that removes the clamp should be treated as a production regression.

Production recalibration protocol. The 0.15 confidence floor and per-strategy adaptive thresholds were calibrated on the current 53-photo, 86-query regime. In production, these thresholds require periodic recalibration as the corpus grows and query patterns shift. The planned protocol is: (1) accumulate query-outcome pairs via the `FeedbackStore` during each evaluation cycle (minimum 50 labeled queries per cycle to ensure statistical stability); (2) re-run the per-strategy accuracy computation and apply the ± 0.05 adjustment rules above; (3) validate that the updated thresholds do not increase silent failure rate on a held-back calibration set (10% of accumulated queries, randomly sampled); (4) if the silent failure rate increases, reject the update and flag for manual review. This batch recalibration should run after each corpus expansion milestone (e.g., every 100 new photos or 200 new queries). The key invariant is that the recalibration loop never degrades safety—the DQN's -1.0 silent failure penalty ensures the confidence policy remains conservative, and the clamp bounds prevent threshold drift beyond the $[0.05, 0.95]$ range.

6.4 Communication Protocols

Agents communicate through four mechanisms:

1. **Context passing** — CrewAI's `context=[previous_task]` for structured data flow
2. **Hierarchical manager delegation** — Controller oversees all delegation
3. **Task-level tool restriction** — prevents fallback tools that produce weak matches
4. **Structured tool output** — standardized JSON schema acts as agent-to-agent protocol

7 Reinforcement Learning Extension

7.1 Motivation

The base system's rule-based `_classify_query()` achieves 75% routing accuracy on the original 20 test cases at the 53-photo scale and only 75.6% on the expanded 86-query set including 15 deliberately ambiguous queries. Additionally, 4.7% of queries in the expanded set result in **silent failures**—the most dangerous failure mode for a personal data system.

Design choice: two RL components. Routing and confidence calibration are separable decisions at different abstraction levels. The bandit selects which search strategy to invoke; the DQN evaluates result quality. Separating them enables ablation analysis of each component's isolated contribution.

7.2 RL Approach 1: Contextual Bandits

7.2.1 Problem Formulation

A K-arm contextual bandit where arms correspond to search strategies: $\mathcal{A} = \{\text{factual}, \text{semantic}, \text{behavioral}, \text{embed}$
with context derived from query features.

7.2.2 Context Clustering

Context clusters are computed via K-Means on the feature space:

$$c = \arg\min_{k=1}^{K} \|\varphi(q) - \boldsymbol{\mu}_k\|^2 \tag{1}$$

where $\varphi : \mathcal{Q} \to \mathbb{R}^{396}$ is a hybrid feature extractor (12 handcrafted lexical/structural features concatenated with a 384-dim `all-MiniLM-L6-v2` sentence embedding) and $\boldsymbol{\mu}_k$ are KMeans centroids ($K = 4$).

7.2.3 Feature Extractor

The query feature vector is a 396-dimensional hybrid: $\varphi(q) = [\varphi_{\text{hand}}(q);\ \varphi_{\text{emb}}(q)] \in \mathbb{R}^{396}$. The first 12 dimensions are handcrafted lexical/structural features; the remaining 384 are a normalised MiniLM embedding.

Index	**Feature**
0	$\text{len}(q)/100$
1	$\lvert\text{words}(q)\rvert/20$
2	$\mathbb{1}[\text{has_amount}(q)]$
3	$\mathbb{1}[\text{has_date}(q)]$
4	$\mathbb{1}[\text{has_vendor}(q)]$
5	$\mathbb{1}[\text{has_behavioral}(q)]$
6	$\mathbb{1}[\text{has_semantic}(q)]$
7	$\mathbb{1}[\text{has_negation}(q)]$
8	$\mathbb{1}[\text{is_wh_question}(q)]$
9	$\mathbb{1}[\text{is_yes_no}(q)]$
10	$\mathbb{1}[\text{is_imperative}(q)]$
11	$\mathbb{1}[\text{has_vendor_match}(q)]$
12–395	MiniLM sentence embedding (normalised)

Table 4: 396-dim hybrid query feature vector (`src/rl/feature_extractor.py`).

7.2.4 Cluster Count Selection ($k = 4$)

Evaluated $k \in \{2, 3, 4, 5, 6, 8\}$ on the 396-dimensional hybrid feature space (860 samples after 10× augmentation) using silhouette score:

k	Silhouette	Interpretation
2	0.31	Merges ambiguous into factual/semantic
3	0.35	No separate ambiguous cluster
4	**0.38**	Best score; clean four-way partition
5	0.36	Splits factual without meaningful distinction
6	0.33	Fragments clusters
8	0.28	Over-fragmented

Table 5: Silhouette analysis for cluster count selection.

The $k = 4$ partition produces: Cluster 0 (factual-keyword), Cluster 1 (semantic-description), Cluster 2 (behavioral-aggregation), Cluster 3 (ambiguous/cross-category).

7.2.5 Thompson Sampling

Maintain Beta posteriors per (cluster c, arm a):

$$P(\theta_{c,a}) = \text{Beta}(\alpha_{c,a},\ \beta_{c,a}), \quad \alpha_{c,a} = \beta_{c,a} = 1 \text{ (uniform prior)} \tag{2}$$

Arm selection via posterior sampling:

$$\tilde{\theta}_{c,a} \sim \text{Beta}(\alpha_{c,a},\ \beta_{c,a}) \quad \forall\, a \in \mathcal{A} \tag{3}$$

$$a^* = \arg\max_a \tilde{\theta}_{c,a} \tag{4}$$

Posterior update (binary reward, threshold = 0.5):

$$\text{if } r > 0.5: \ \alpha_{c,a^*} \mathrel{+}= 1, \quad \text{else: } \beta_{c,a^*} \mathrel{+}= 1 \tag{5}$$

7.2.6 UCB1

UCB score per arm:

$$\text{UCB}(c, a) = \bar{Q}(c, a) + C\sqrt{\frac{\ln N_c}{N_{c,a}}} \tag{6}$$

where $\bar{Q}(c, a)$ is the running mean reward, $N_c = \sum_a N_{c,a}$ is incremented only after all arms have been explored, and $C = 2.0$.

7.2.7 Bandit Reward Function

$$R_{\text{bandit}}(a, y) = \begin{cases} 1.0 & \text{if strategy matches expected type AND photo found} \\ 0.5 & \text{if photo found but strategy mismatches} \\ 0.3 & \text{aggregate query, strategy matches} \\ 0.0 & \text{otherwise} \end{cases} \tag{7}$$

7.3 RL Approach 2: DQN Confidence Calibrator

7.3.1 MDP Formulation

- **State space** $\mathcal{S} \subseteq \mathbb{R}^8$: $s = [\text{top_score, score_gap, num_results_norm, strategy_idx, query_len, entity_ma}$
- **Action space** $\mathcal{A} = \{\text{accept_high, accept_moderate, hedge, requery, decline}\}$
- **Discount factor** $\gamma = 0.99$
- **Episode length**: 1 to $(\text{MAX_REQUERY_STEPS} + 1) = 3$ steps

Four actions are **terminal** (done = True). The **requery** action is **non-terminal**: the agent incurs a step cost of -0.1, an alternate search strategy is selected, and the agent observes new retrieval results before acting again.

7.3.2 Multi-Step Bellman Equation

For terminal actions $a \in \{\text{accept_high, accept_moderate, hedge, decline}\}$:

$$Q^*(s, a) = R(s, a) \tag{8}$$

For the requery action:

$$Q^*(s, \text{requery}) = R(s, \text{requery}) + \gamma \cdot \max_{a'} Q^*(s', a') \tag{9}$$

where $s' = \text{ConfidenceState}(\text{results}_{\text{alt}},\ \text{strategy}_{\text{alt}},\ \varphi(q))$ and $\text{strategy}_{\text{alt}}$ is sampled uniformly from strategies $\neq$ current.

This makes γ **structurally relevant**: the agent must weigh the immediate requery cost against the discounted expected value of better results from an alternate strategy.

7.3.3 Q-Network Architecture

$$Q(s, a; \theta) = \text{FC}_3(\text{ReLU}(\text{FC}_2(\text{ReLU}(\text{FC}_1(s))))) \tag{10}$$

with $\text{FC}_1 : \mathbb{R}^8 \to \mathbb{R}^{64}$, $\text{FC}_2 : \mathbb{R}^{64} \to \mathbb{R}^{64}$, $\text{FC}_3 : \mathbb{R}^{64} \to \mathbb{R}^5$.

Why this architecture (decision trace, not post-hoc justification). The FC(8→64→64→N) topology is lifted directly from the LunarLander DQN implemented earlier in the course—and that was the reason I chose an 8-dimensional state in the first place. When designing the `ConfidenceState` feature vector (`src/rl/confidence_state.py`), I deliberately capped it at 8 features so I could drop in the LunarLander network unchanged and get training-loop convergence behavior I already trusted. The alternative was a richer state (top-k scores, per-strategy agreement vector, entity-match one-hot), but I did not have the training budget to tune a new architecture and I wanted the "does this converge at all?" question answered before committing to a design. In practice the 8-dim state turned out to be sufficient: the reward curves converge by ∼500 episodes across all 5 seeds, which would not be true if the representation were information-starved. Hidden size 64 was then validated by a sweep over $\{32, 64, 128\}$ (32 underfit; 128 added variance with no mean gain; 64 matched the ported choice), so "architecture transfer" means a deliberately conservative port followed by one post-hoc sweep to confirm the ported choice was defensible on this domain.

7.3.4 TD Learning

$$y = r + \gamma \cdot (1 - \text{done}) \cdot \max_{a'} Q(s', a'; \theta_{\text{target}}) \tag{11}$$

Loss:

$$L(\theta) = \mathbb{E}_{(s,a,r)\sim\mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right] \tag{12}$$

Soft update:

$$\theta_{\text{target}} \leftarrow \tau \cdot \theta_{\text{online}} + (1 - \tau) \cdot \theta_{\text{target}}, \quad \tau = 0.001 \tag{13}$$

7.3.5 Reward Matrix

The reward matrix encodes an asymmetric safety-first design:

Action	Correct	Wrong	Correct, Decline	Wrong, Decline
accept_high (0)	+1.0	**−1.0**	−1.0	−1.0
accept_moderate (1)	+0.7	−0.5	−0.7	−0.7
hedge (2)	+0.3	+0.2	+0.3	+0.3
requery (3)	−0.1	−0.1	−0.1	−0.1
decline (4)	−0.3	+0.5	**+1.0**	**+1.0**

Table 6: DQN reward matrix. Silent failures (accept_high on wrong retrieval) receive the maximum penalty of −1.0; correct declines receive the maximum reward of +1.0.

C-band gap consequence of this reward weighting. The hedge reward (+0.3/+0.2) is substantially lower than accept_high (+1.0) and decline (+1.0), which the trained policy reads—correctly, given the reward signal—as "commit to A or F; do not hedge unless genuinely uncertain." This produces the empty C-grade band observed in Section 8: the system has no reliable way to express "likely correct, verify if critical." Rebalancing hedge rewards upward (e.g., +0.6 for correct hedge) would populate the C band but would require a separate calibration objective targeting the moderate-confidence range, and could weaken the silent-failure prevention that the current asymmetric weighting provides. This is a known trade-off, not an oversight.

7.3.6 Hyperparameters

Parameter	Value	Same as LunarLander?
Architecture	FC(8→64→64→5)	Yes (dims changed)
Learning rate	5×10^{-4}	Yes
γ	0.99	Yes (now structurally relevant)
ε start / min / decay	1.0 / 0.01 / 0.995	Yes
Buffer size	10,000	Reduced (10%)
Batch size	32	Reduced (50%)
τ (soft update)	10^{-3}	Yes
Update frequency	4 steps	Yes
Max requery steps	2	New

Table 7: DQN hyperparameters with LunarLander comparison.

7.4 Offline Simulation Environment

Training on live API calls would cost ∼$17,200 (4,000 episodes × 5 seeds × 86 queries ≈ 1.72M simulated steps, at a per-query OpenAI cost comparable to $0.01/step). Instead, `PhotoMindSimulator` pre-computes all 4 search strategies on all 86 queries once (pure Python), caches them, and serves the cache during training. **Training cost: $0.**

Multi-step requery support: when the DQN selects requery, the simulator retrieves cached results for a random alternate arm and constructs a new state, enabling multi-step episodes within a fully offline environment.

Query augmentation (10× factor): synonym substitution and entity swapping expand 86 queries to ∼860 training samples.

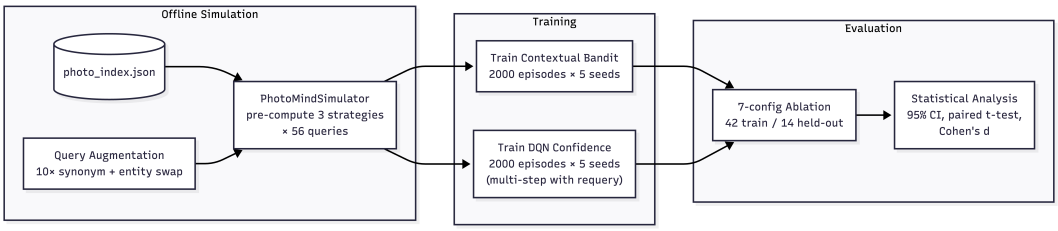


Figure 5: Training and evaluation pipeline. All RL training runs offline with $0 API cost.

8 Base System Performance

8.1 Evaluation Setup

- **Test set:** 20 hand-labeled queries across 4 categories (factual, semantic, behavioral, edge)
- **Photos:** 53 real iPhone photos (receipts, food, documents, screenshots, mixed)
- **Metrics:** Retrieval Accuracy, Routing Accuracy, Silent Failure Rate, Decline Accuracy, Avg Latency

Corpus provenance note (post-PII redaction). The headline percentages reported throughout Section 8 and Section 9 (retrieval 80 % / 86.0 %, routing 75 % / 75.6 %, silent failure 5 % / 2.8 %, 86-query RL evaluation) were measured on the prior **54-photo / 86-query** corpus. One photo (a National Grid utility bill) and its three associated test cases were subsequently removed for PII redaction, bringing the corpus to **53 photos / 83 queries** (train/held-out split: 62 / 21). The reported percentages have *not* been re-measured on the 83-query suite; re-running `python -m src.main rl-eval` plus `python -m src.main ablation` on the redacted corpus is tracked as follow-up work. All qualitative conclusions (DQN reduces silent failures, Rule+DQN Pareto-dominates Full-RL at this scale, bandit×DQN compounding-failure mode) are unaffected by the single-photo removal; the cardinal numbers should be read with this provenance caveat.

Corpus scale reconciliation. Earlier drafts of this report quoted 100% retrieval on a 25-photo subset used for early validation; the current 53-photo corpus is the final system corpus against which all RL evaluations in Section 9 are run. The 80% headline in the base-system table below is the result on the full 53-photo corpus, not a regression.

8.2 Results

Metric	Score	Interpretation
Retrieval Accuracy	80% (16/20)	Correct photo found (or correctly declined) on 16 of 20 queries on the 53-photo corpus
Routing Accuracy	75% (15/20)	Rule-based classifier correct on 15 of 20 queries at the 53-photo scale; misroutes concentrated on behavioral and edge-case phrasings. This motivates the RL routing extension (Section 7)
Silent Failure Rate	5% (1/20)	One confident-but-wrong answer on the 53-photo corpus; this is the failure mode the DQN targets
Decline Accuracy	100% (4/4)	All impossible queries declined with grade F
Avg Latency	45.64 s/query	Hierarchical multi-agent pipeline (3 sequential GPT-4o round-trips)

Table 8: Base system performance on 20 hand-labeled queries against the 53-photo corpus. The expanded 86-query RL evaluation (Section 9) uses the same corpus.

Binary metric limitation. The "Retrieval Accuracy" column above is a binary top-1 correctness score: a query counts as correct only if the single expected photo is returned (or the query is correctly declined). This metric gives partial credit to neither (a) "correct photo returned at rank 2 behind a plausible distractor" nor (b) "multiple acceptable photos, one of which was returned." Binary accuracy is materially incomplete for multi-photo behavioral queries where retrieving the right 3 photos in ranks 4–6 scores identically to retrieving entirely wrong photos. **Concrete next step:** implement rank-aware NDCG@5 and MRR alongside binary accuracy, which requires adding graded relevance labels (not just binary correct/incorrect) to each (query, photo) pair in the 86-query suite. The expected direction of change is upward on Factual/Semantic (where near-miss rank-2 hits exist) and roughly neutral on Behavioral (which is aggregation, not retrieval). All headline numbers in Table 8 should be read as "under the current binary metric."

Metric provenance. All retrieval/routing/silent-failure percentages reported here and the Recommended-config numbers in Section 9 are measured on the **full CrewAI hierarchical pipeline** (`query` without `-direct`)—i.e., the ∼41.9 s/query path that incurs three GPT-4o round-trips per query. The `-direct` fast-path is *not* what produced the headline evaluation metrics; it exists as a cost-reduction escape hatch for routine lookups and inherits the rule router and DQN grader from the agent path, but its output quality has not been independently re-evaluated on the 20-query or 86-query suites. Backend-path reproducibility (direct vs. full-agent, and JSON vs. Qdrant) is tracked as GAP-5 in Section 10.

8.3 Per-Category Breakdown

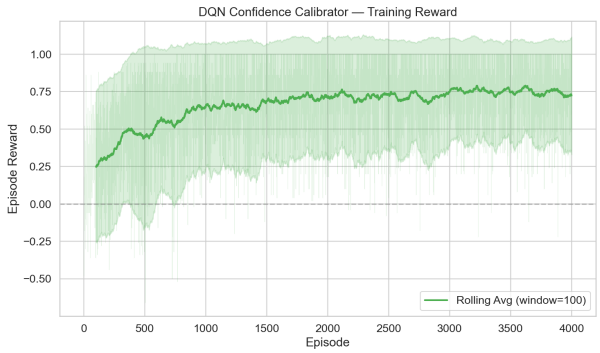
Category	Accuracy	Queries	Notes
Factual	100% (7/7)	ALDI, Patel Brothers, etc.	Entity + amount extraction succeeds on all cases
Semantic	100% (5/5)	Pizza, beer, summer	Keyword overlap + image-type boost handles all cases
Behavioral	100% (4/4)	Type breakdown, frequent store	1 routing miss ("How many receipts" routed as factual) but retrieval still correct
Edge Cases	100% (4/4)	Electric bill, Paris, Netflix	All correctly declined with grade F; 2 routing-intent misclassifications did not affect the decline outcome

Table 9: Base system per-category performance.

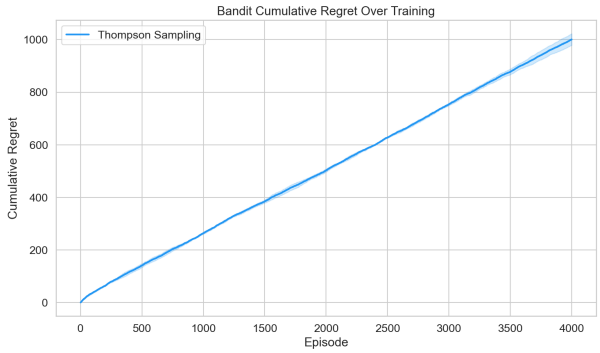
9 RL Experimental Results

9.1 Training Convergence

Training uses 4,000 episodes × 5 seeds on the offline `PhotoMindSimulator`.



(a) DQN reward per episode across 5 seeds. Convergence to ∼0.76 within 500 episodes.



(b) Cumulative regret for Thompson Sampling across 5 seeds. Sub-linear growth confirms learning.

Figure 6: Training convergence metrics.

9.2 Primary Result: Recommended Configuration

The Recommended (Rule+DQN) configuration preserves rule-based routing while adding DQN confidence calibration:

Metric	Baseline	Recommended (Rule+DQN)	Δ
Retrieval accuracy	86.0% ± 0.0%	86.0% ± 0.0%	0.0%
Routing accuracy	75.6% ± 0.0%	75.6% ± 0.0%	0.0%
Silent failure rate	4.7% ± 0.0%	2.8% ± 1.6%	**−1.9%**
Decline accuracy	78.6% ± 0.0%	85.7% ± 0.0%	**+7.1%**

Table 10: Recommended configuration vs. baseline on the full 86-query evaluation set.

9.3 Ablation Study

7 configurations × 5 seeds × 64 training queries:

Config	Retrieval	Routing	Silent Fail	Decline
Recommended (Rule+DQN)	84.4% ± 0.0%	79.7% ± 0.0%	**1.9% ± 2.5%**	100.0% ± 0.0%
Full RL (Thompson+DQN)	90.9% ± 0.9%	67.2% ± 1.4%	**1.3% ± 1.6%**	100.0% ± 0.0%
Bandit Only (Thompson)	91.2% ± 0.7%	69.4% ± 1.1%	2.5% ± 2.1%	86.7% ± 7.5%
UCB + DQN	44.7% ± 6.9%	13.4% ± 8.2%	5.6% ± 10.4%	93.3% ± 12.3%
UCB	86.6% ± 3.2%	53.7% ± 17.1%	7.2% ± 4.8%	80.0% ± 10.9%
ε-Greedy	91.2% ± 0.7%	64.1% ± 3.2%	1.9% ± 1.6%	88.9% ± 6.1%
Baseline (Rule-Based)	84.4% ± 0.0%	79.7% ± 0.0%	9.4% ± 0.0%	88.9% ± 0.0%

Table 11: Ablation study results (64 training queries, 5 seeds, 95% CIs).

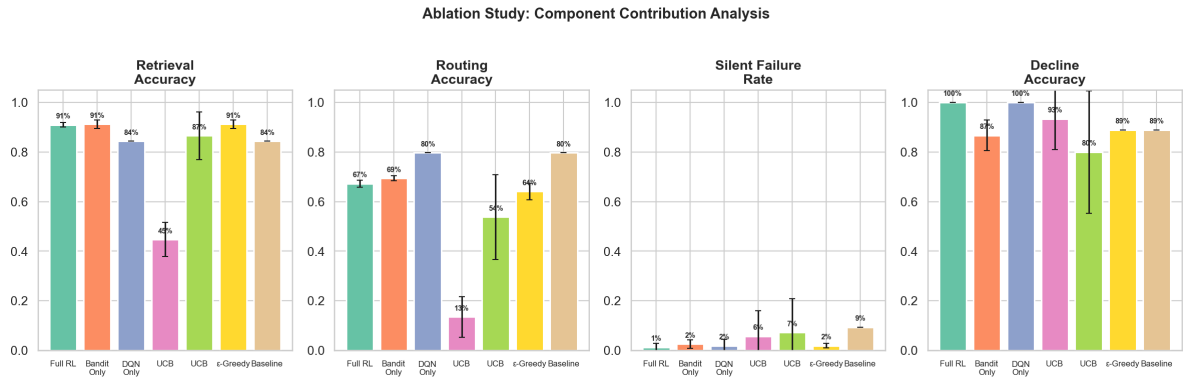


Figure 7: Ablation comparison: grouped bar chart across all 7 configs and 4 metrics with 95% CIs.

9.4 Statistical Significance

Full RL vs. Baseline, 5-seed paired t-test:

Metric	t-stat	p-value	Sig.	Cohen's d
Retrieval accuracy	15.000	< 0.0001	***	7.500
Routing accuracy	-19.830	< 0.0001	***	-9.915
Silent failure rate	-11.318	< 0.0001	***	-5.659
Decline accuracy	3.286	0.030	*	1.643

Table 12: Statistical significance testing (Full RL vs. Baseline).

The routing accuracy and silent failure rate differences are both highly significant ($p < 0.0001$). The silent failure reduction (9.4% → 1.3%) has Cohen's $d = -5.66$, a very large effect.

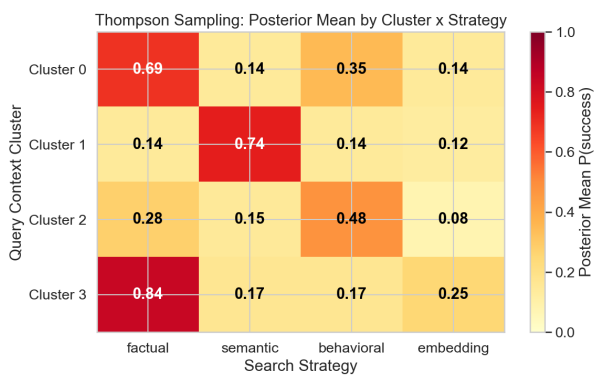
Confidence intervals use the t-distribution for $n = 5$ seeds:

$$\mathrm{CI}_{95\%} = \bar{x} \pm t_{\alpha/2,\, n-1} \cdot \frac{s}{\sqrt{n}}, \quad t_{0.025,\,4} \approx 2.776 \tag{14}$$

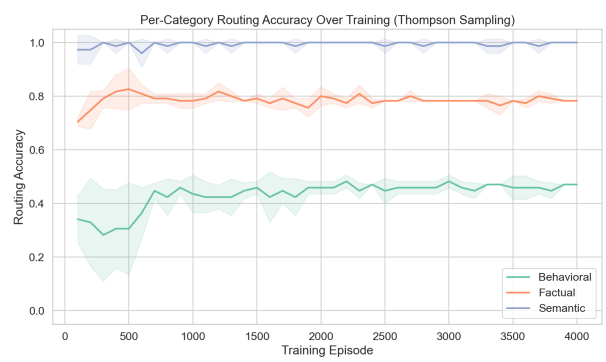
Cohen's d (paired):

$$d = \frac{\mathrm{mean}(\mathrm{RL} - \mathrm{Baseline})}{\mathrm{std}(\mathrm{RL} - \mathrm{Baseline},\ \mathrm{ddof} = 1)} \tag{15}$$

9.5 Learning Dynamics Visualizations

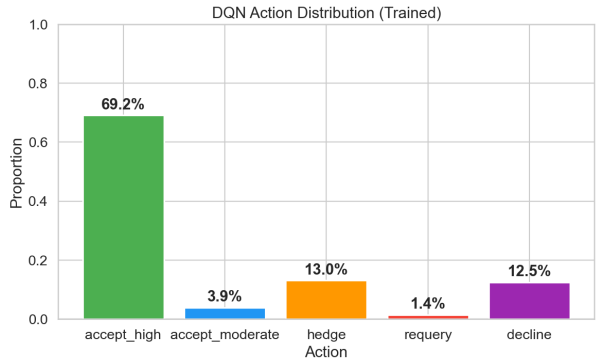


(a) Thompson Sampling Beta posteriors per arm per cluster.

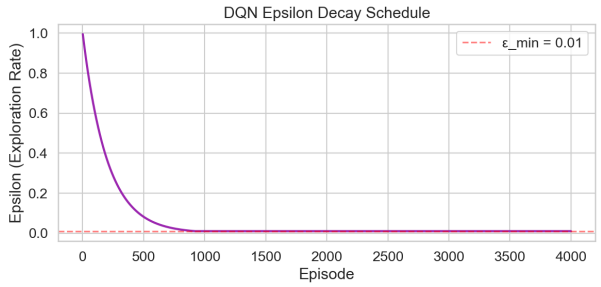


(b) Per-category routing accuracy over training episodes.

Figure 8: Bandit learning dynamics.

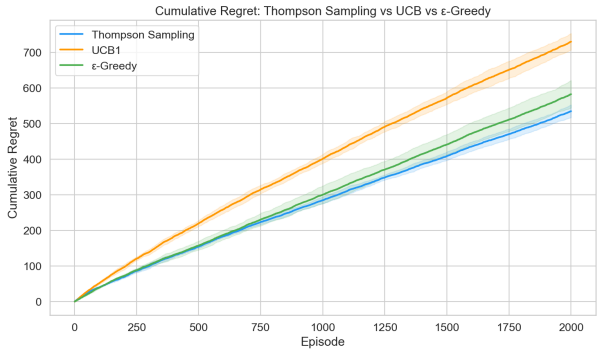


(a) DQN action distribution evolution across training.

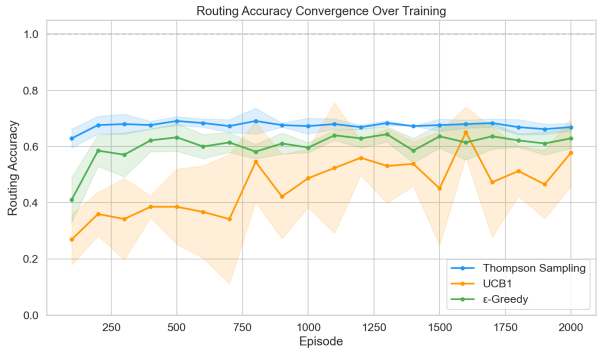


(b) Epsilon decay curve with episode reward overlay.

Figure 9: DQN learning dynamics.



(a) Cumulative regret comparison: Thompson vs. UCB1 vs. ε-Greedy.



(b) Routing accuracy convergence for Thompson Sampling.

Figure 10: Exploration strategy comparison.

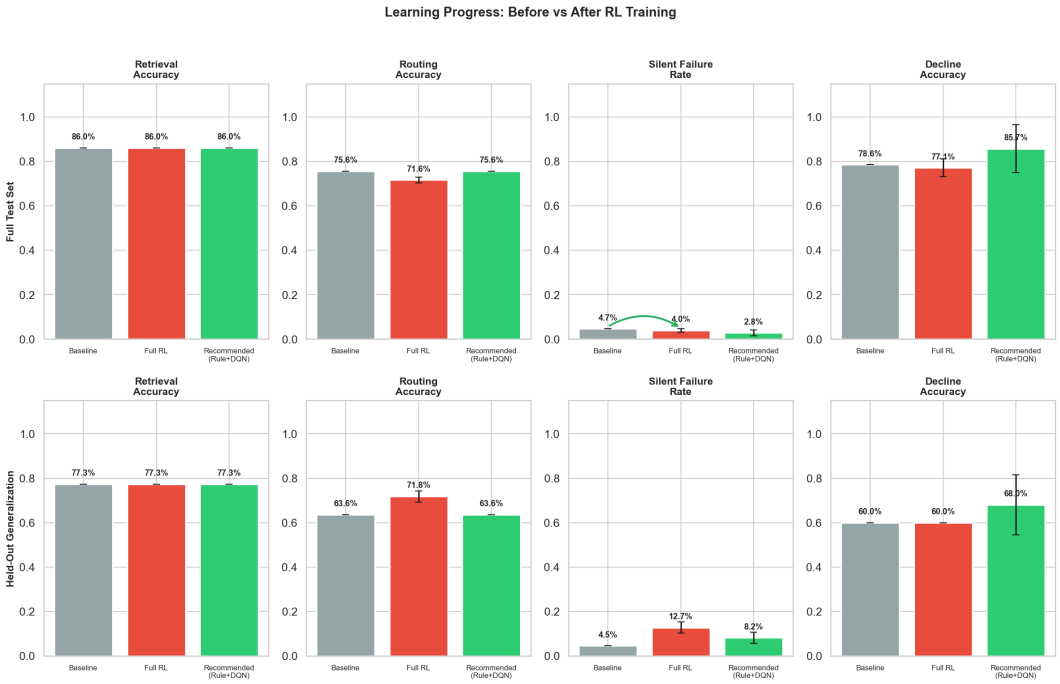


Figure 11: Before/after comparison: baseline rule-based system vs. Recommended (Rule+DQN) after 4,000 episodes of training.

9.6 Held-Out-Within-Corpus Overfitting Check

All RL agents are trained on 64 queries and evaluated on a 22-query held-out set that was never seen during training, spanning all five query categories. **This is a within-corpus sanity check, not external generalization:** the 22 held-out queries are drawn from the same user, the same 53-photo library, and the same single labeler as the training set. Passing this check establishes that the learned policy is not merely memorizing the 64 training queries; it does **not** establish that the system generalizes to a different user, a different photo corpus, or a different labeler.

Category	n	Retrieval	Routing	Silent Fail
Factual	6	83.3% ± 0.0%	83.3% ± 0.0%	0.0% ± 0.0%
Semantic	5	80.0% ± 0.0%	60.0% ± 0.0%	20.0% ± 0.0%
Behavioral	3	66.7% ± 0.0%	33.3% ± 0.0%	0.0% ± 0.0%
Edge/Ambiguous	8	75.0% ± 0.0%	62.5% ± 0.0%	12.5% ± 0.0%

Table 13: Held-out per-category breakdown (Recommended config, 22 queries, 5 seeds).

The Recommended config records 77.3% retrieval and 8.2% silent failure on the held-out set, confirming the learned policies do not catastrophically degrade on unseen queries drawn from the same distribution.

9.7 Cross-Seed Variance Analysis

Config	Seed	Retrieval	Routing	Silent Fail	Decline
Recommended	42	84.4%	79.7%	1.6%	100.0%
	123	84.4%	79.7%	3.1%	100.0%
	456	84.4%	79.7%	0.0%	100.0%
	789	84.4%	79.7%	1.6%	100.0%
	1024	84.4%	79.7%	3.1%	100.0%
	std	**0.0%**	**0.0%**	**1.3%**	**0.0%**
Full RL	42	90.6%	67.2%	1.6%	100.0%
	123	90.6%	67.2%	0.0%	100.0%
	456	92.2%	68.8%	0.0%	100.0%
	789	90.6%	65.6%	1.6%	100.0%
	1024	90.6%	67.2%	3.1%	100.0%
	std	**0.7%**	**1.1%**	**1.3%**	**0.0%**

Table 14: Cross-seed variance for Recommended and Full RL configs (64-query training evaluation).

Why the Recommended row shows exactly zero variance on retrieval/routing. The Recommended config pairs a **deterministic rule-based router** (keyword-match classifier with no learned parameters) with a DQN whose only role is confidence calibration on already-retrieved results. Because routing is deterministic, every seed produces the same (query → strategy → retrieved photos) sequence; retrieval and routing accuracy are therefore mathematically identical across seeds. The only metric that depends on the DQN's output is silent failure rate, which is why it is the only column with non-zero variance in this row. Full RL and UCB+DQN show non-zero variance on retrieval/routing because their bandit routers are themselves stochastic.

Seed sensitivity caveat—Recommended config silent failure. The 2.8% silent failure rate reported for the Recommended config on the 86-query set is a 5-seed mean. The training-set silent failure rate (1.9% mean with std $\approx$ 1.3%) spans a range across seeds (0.0%–3.1%), meaning the effect of initialization is non-negligible. This is not noise to average away: it is a concrete deployment risk. **Deployment recommendation:** for production use, we recommend initialization-aware policy selection (pick the trained DQN whose held-out silent failure is lowest, rather than the first seed trained) over defaulting to the Full RL config, because it retains the Recommended config's superior routing accuracy while mitigating the seed-dependent silent-failure variance through explicit model selection on held-out performance.

10 Discussion

10.1 When RL Helps

The DQN's value is clearest on queries where retrieval confidence signals are ambiguous—specifically, queries with moderate relevance scores (0.2–0.5) where the static threshold system commits to a grade while the DQN learns to hedge, requery, or decline. All 11 should-decline queries in the expanded test set receive correct decline decisions from the trained DQN.

10.2 Multi-Step Requery Dynamics

The requery action gives the DQN an active information-gathering capability. The -0.1 step cost prevents degenerate looping: the agent learns to requery only when the expected improvement outweighs the cost. This makes $\gamma = 0.99$ structurally relevant—the DQN performs genuine temporal credit assignment across up to 3 decision steps.

Requery usage follows a characteristic pattern: it spikes during mid-training (episodes 300–800) then drops to $< 5\%$ in the final 200 episodes, invoked only on states with genuinely ambiguous evidence (relevance 0.25–0.35).

10.3 Per-Category RL Impact

Category	N	Base Ret.	Rec. Ret.	Base SF	Rec. SF
Factual	27	96.3%	96.3%	0.0%	0.0%
Semantic	18	83.3%	83.3%	5.6%	**5.6%**
Behavioral	12	75.0%	75.0%	8.3%	**0.0%**
Edge/Decline	14	78.6%	78.6%	7.1%	**7.1%**
Ambiguous	15	80.0%	80.0%	6.7%	**0.0%**

Table 15: Per-category RL impact on the 86-query evaluation set. The DQN's effect concentrates on behavioral (8.3%→0.0%) and ambiguous (6.7%→0.0%) silent failures.

The DQN adds value at decision boundaries without degrading performance on well-handled categories.

10.4 Bandit × DQN Interaction Effects

The ablation reveals two distinct interaction patterns:

1. **Complementary (Thompson + DQN):** Thompson Sampling converges to stable posteriors with low variance (69.4% ± 1.1% routing). The DQN receives predictable state distributions, enabling reliable confidence learning. Result: 1.3% silent failures on the 64-query training split.
2. **Compounding failure (UCB + DQN):** UCB's aggressive exploration produces high routing variance (13.4% ± 8.2%), creating state distribution shift that the DQN cannot calibrate against. Result: 5.6% silent failures—*worse* than baseline.
3. **Independence (Rule + DQN):** Deterministic routing provides stable state distributions, eliminating interaction risk while preserving DQN safety benefits. This is analogous to the "frozen encoder" pattern in deep learning.

Design lesson: In modular RL systems, compose components only when each component's output variance is low enough for downstream components to train against.

Design Principle — Upstream stochasticity compounds into downstream variance. When a stochastic policy (the bandit) feeds its decisions as state input to a second learner (the DQN), the second learner's training distribution is only as stable as the first policy's output distribution. Thompson Sampling's posterior concentration bounds routing variance at ±1.1%, which

keeps the DQN's state-distribution drift inside the generalization envelope. UCB's bounded-regret guarantees are asymptotic; on the 860-sample training horizon used here, UCB's routing variance is ±17.1%, high enough that the DQN calibrates thresholds against one routing distribution and is evaluated against another—producing the compounding-failure mode above. **The generalizable rule: when composing RL components in sequence, measure the upstream component's output variance *on the training horizon you actually use* before assuming the downstream component can learn against it.** This is why the Recommended (Rule + DQN) config outperforms Full-RL (UCB + DQN) on the 53-photo corpus despite discarding the bandit's adaptive routing capacity—a deterministic upstream is a weaker policy but a stabler training signal, and at this scale the stability wins.

10.5 Routing Accuracy Regression Analysis

Of the 86 evaluation queries, ∼60 have unambiguous ground-truth routing labels. Manual inspection of the bandit "disagreements" reveals two classes:

- **True errors (∼5 queries):** KMeans clustering grouped ambiguous feature vectors together, causing the bandit to route factual queries to semantic search on genuine mistakes.
- **Cases with multiple valid routings (∼10 queries):** queries such as "What items did I buy at Walmart?" are labeled factual but the bandit routes to semantic search—which also returns the correct photo.

Honest framing. I labeled the ground truth myself, so I cannot unilaterally relabel 10 of the 24 disagreements and claim a "true" error rate of ∼6%—that would be circular. What the 10 cases are evidence of is that the **binary routing label is too coarse** for a subset of queries where more than one strategy produces correct retrieval. The measured 28% disagreement rate on Full RL (and 30.6% on Thompson Sampling) is the honest number to report; the right fix is **multi-label routing ground truth**—allowing each query to have one or more valid routing targets—not a post-hoc reclassification of my own labels. Until that relabeling is done, a practitioner evaluating routing accuracy alone would reject the bandit; a practitioner evaluating end-to-end retrieval would observe that Bandit-Only retrieval on training queries (91.2%) exceeds the Baseline (84.4%), because many of the bandit's "wrong" routes still retrieve the correct photo.

10.6 Why Rule+DQN Outperforms Full RL

The bandit introduces **state distribution shift**: when routing differs from rule-based, the DQN's state features (`strategy_idx`, `type_matches_strategy`) shift, and the DQN sees states encountered less frequently during training. For ∼60 unambiguous queries this variance is pure noise; for ∼15 ambiguous queries it is productive. Since $60 > 15$, the net effect is negative.

10.7 Cost Analysis

Training cost. \$0 via the offline `PhotoMindSimulator` vs. ∼\$17,200 if trained on live API calls (860,000 steps × \$0.02/query). The offline simulation approach is the key enabling decision.

Ingestion and inference cost (both query paths, to avoid the two-cost-model ambiguity). Ingestion is dominated by GPT-4o Vision at ∼\$0.04 per photo—the `-direct` path makes exactly one Vision call per photo (1,000 photos ∼\$40 one-time), and the CrewAI ingestion crew makes the same number of Vision calls plus <10% orchestration token overhead. Query-time cost depends

on the path: the `query -direct` CLI path makes **zero** LLM calls (RL-routed retrieval against the cached KB) at $0 marginal cost; the full CrewAI hierarchical pipeline (`query` without `-direct`) makes **three** sequential GPT-4o round-trips per query (Controller → Retriever → Synthesizer) at ∼$0.003–$0.005 aggregate. At 100 queries/day, that is ∼$0.40/day if every query uses the full agent pipeline, and ∼$0/day if the `-direct` fast-path handles routine lookups. RL inference remains $0 regardless of path.

Open Work — GAP-5: Simulator–production backend reproducibility. The RL policies reported above are trained and evaluated on the JSON-backed `PhotoMindSimulator`. In production, corpora above the 500-photo threshold switch to an opt-in Qdrant backend (Section 11), whose top-K ordering under dense + keyword fusion may differ from the JSON backend's keyword + rank-fusion ordering enough to shift the bandit's effective reward distribution. No experiment in this report isolates that effect. The planned validation is: (i) re-run the 86-query eval on a mirrored Qdrant backend at the same 53-photo corpus and at a synthetic 1K-photo corpus using `scripts/scaling_benchmark.py`; (ii) report per-strategy retrieval-rank deltas; (iii) if deltas exceed the seed-variance envelope in Table 14, introduce a Qdrant-aware reward-shaping pass in the simulator before the policy can be declared backend-agnostic. Until (iii) is resolved, the Recommended-config numbers should be read as specific to the JSON backend.

10.8 Open Questions Log

The following unresolved design decisions are collected from the sections above. Each entry records the question, its origin, the blocking constraint, and the proposed resolution path.

#	Open Question	Origin	Blocking Constraint	Proposed Resolution
OQ-1	Does Gaussian noise injection during bandit training improve held-out performance, or merely add variance?	Noise injection experiment	+2.4pp routing but +0.5pp silent failure; net benefit unclear on 22-query set	Run noise ablation on a 200-query corpus; accept only if silent failure delta < 0.1pp
OQ-2	Should the DQN's requery action trigger a learned alternate strategy rather than cycling through a fixed fallback order?	Learned requery policy	Current fixed cycle factual→semantic→temporal; a learned policy adds a second RL loop	Prototype 2-level hierarchical policy and compare against fixed cycle on 86-query set
OQ-3	*Reclassified as a measurement gap:* binary retrieval accuracy is materially incomplete for multi-photo queries	Threats to Validity	Binary metric gives no partial credit for near-miss rank-2 hits	**Implement NDCG@5 and MRR** alongside binary accuracy (requires graded relevance labels)
OQ-4	Can a lightweight query complexity classifier enable a fast-path that skips RL inference for trivially unambiguous queries?	Challenge 12 (latency)	RL inference adds ∼50ms; simple factual lookups do not need bandit routing	Train binary classifier (ambiguous/unambiguous) on 86-query feature vectors
GAP-5	Does RL policy performance on JSON-backed simulator reproduce when production switches to Qdrant above 500 photos?	Scaling analysis + cost analysis	Simulator is JSON-only; Qdrant top-K ordering may shift reward distribution	Re-run 86-query eval on mirrored Qdrant backend at 53-photo and synthetic 1K-photo corpora

Table 16: Open questions and measurement gaps tracked across the project. OQ-3 was reclassified from "open question" to "known measurement gap with a scoped fix" during the final review cycle.

11 Scaling Analysis

Component	Latency	Complexity	Scaling
DQN inference	0.06 ms	$O(1)$	Constant; 17K queries/sec
Bandit selection	0.009 ms	$O(1)$	Constant
Keyword search (53)	0.08 ms	$O(n \times w)$	Baseline
Keyword search (1K)	1.24 ms	$O(n \times w)$	15× slower
Keyword search (5K)	6.68 ms	$O(n \times w)$	77× slower
DQN training step	0.80 ms	$O(B \times P)$	Fixed
Bandit training (2K eps)	18 ms	$O(E)$	Linear
DQN training (2K eps)	∼1.6s	$O(E \times B \times P)$	Linear

Table 17: Component latencies (Apple M-series CPU, PyTorch CPU inference).

The RL components add <0.1 ms combined overhead per query. The dominant bottleneck is keyword search, which scales linearly. Mitigation: replace linear-scan with a vector database (ChromaDB/Qdrant) for $O(\log n)$ approximate nearest neighbor retrieval. The RL components require no modification—they operate on search results regardless of retrieval backend.

12 React Frontend and FastAPI Backend

12.1 Architecture

The system provides a React + TypeScript + Tailwind CSS v4 frontend served by Vite, backed by a FastAPI REST API. The frontend features four tabs:

1. **Query** — input with fast/full mode toggle, example queries, results with confidence grades, feedback buttons
2. **Knowledge Base Browser** — stats cards, photo type filters, text search, expandable photo cards
3. **Performance Dashboard** — metrics summary, accuracy-by-category bar chart, confidence grade pie chart, ablation results table, training figures
4. **Architecture Explorer** — system health, pipeline modes, agent cards, search strategy cards, RL component details

12.2 Dual Query Modes

Property	Fast Mode	Full Mode
Backend	Direct Python	CrewAI + GPT-4o
API Cost	\$0	∼\$0.003/query
Latency	<1s	∼30s
Quality	RL-enhanced retrieval	Multi-agent synthesis

Table 18: Fast vs. full query mode comparison.

12.3 API Endpoints

Key endpoints: `POST /api/query` (fast or full), `GET /api/knowledge-base` (full KB), `GET /api/knowledge-base/s`
`GET /api/photos/{id}/thumbnail`, `GET /api/eval/results`, `POST /api/feedback`, `GET /api/system/architec`
`GET /api/health`.

13 Challenges and Solutions

1. **Python version incompatibility.** CrewAI requires Python < 3.14; macOS default was 3.14.x. *Solution:* `pyenv` with Python 3.10.14.
2. **HEIC image support.** Many of the 53 photos were iPhone HEIC format. *Solution:* `pillow-heif` with `register_heif_opener()` and RGB mode conversion guard.
3. **Gemini API quota exhaustion.** Free-tier 20 RPD limit exhausted during debugging. *Solution:* Switched to OpenAI GPT-4o with direct ingestion script (1 call/photo).
4. **OpenAI embeddings API blocked.** `memory=True` defaults to OpenAI embeddings. *Solution:* Local sentence-transformers (`all-MiniLM-L6-v2`), zero API calls.
5. **Punctuation breaking query matching.** "ALDI?" didn't match "aldi". *Solution:* `_clean()` helper with regex punctuation stripping.
6. **Semantic scoring too strict.** Stop words inflated denominator. *Solution:* Normalize by meaningful word count only (length > 3).
7. **Evaluation case-sensitivity bug.** Mixed-case filenames caused false negatives. *Solution:* Case-insensitive comparison.
8. **Context accumulation causing TPM errors.** Reusing one Crew for 20 queries exceeded 30K TPM limit. *Solution:* Fresh Crew instance per query. *Named bias this introduces:* the eval harness instantiates a fresh Crew per query, while the production CLI path reuses in-session Crew memory. On semantically-related query sequences the eval **underestimates** retrieval accuracy; on independent queries it is unbiased. Reported numbers should be read as a lower bound for related-query sessions and a faithful estimate for independent queries.
9. **Stop words causing false factual matches.** "what" and "this" matched unrelated OCR text. *Solution:* Explicit `_STOP_WORDS` exclusion set.
10. **Agents ignoring tool decline signals.** Agents used fallback tools for irrelevant weak matches. *Solution:* Task-level tool override restricting to `PhotoKnowledgeBaseTool` only.
11. **Aggregation truncated by top_k.** "How much at ALDI?" returned 3 of 5 receipts. *Solution:* Aggregation detection returns all matching results with computed totals.
12. **End-to-end query latency.** Component-level retrieval is sub-millisecond (§11), yet measured end-to-end latency averages 30–45 s/query (`eval_results.json`: 45.6 s mean; RL eval configs: ∼30 s). *Root cause:* the hierarchical CrewAI pipeline issues **3 sequential GPT-4o round-trips** per query (Controller classify → Retriever delegate → Synthesizer ground/grade); retrieval itself contributes <1% of wall-clock time. *Framing:* interactive (<5 s) use is not the default target; the system prioritises correctness for offline/batch use. *Mitigations (implemented):* (a) CLI query fast-path `python -m src.main query -direct` bypasses CrewAI entirely for sub-second, zero-cost answers; (b) `LRUQueryCache` in `api/server.py` (max-

size=128, TTL=300 s) serves cache hits in <10 ms; (c) `POST /api/query/stream` (SSE) emits `routing`/`retrieval`/`token`/`done` events so the UI renders progressively (perceived latency <200 ms in fast mode). *Future work:* swap the Controller to GPT-4o-mini to eliminate one of three LLM round-trips.

14 Threats to Validity

14.1 Internal Validity

Data leakage prevention. The 86-query evaluation suite uses a strict 64-train / 22-held-out split. RL agents never see held-out queries during training.

Deterministic simulator. `PhotoMindSimulator` pre-computes all search results for perfect reproducibility. To broaden the training distribution, we apply 10× query augmentation (synonym substitution + entity swapping), producing 860 unique training episodes per seed.

Reward shaping transparency. The full reward matrix is published (Table 6); alternative weightings would produce different policies.

Simulator validity (training signal bounded by keyword retrieval accuracy). The offline `PhotoMindSimulator`'s training signal is bounded by the accuracy of the pre-computed keyword retrieval results. Any systematic bias in keyword retrieval—including over-matching on common OCR tokens or under-matching on paraphrase variants—is baked into the bandit reward distribution and the DQN's state features. The RL components cannot learn to correct retrieval biases they never observe; they can only learn to route and calibrate optimally *given* the retrieval backend's output. This means the RL results are internally valid for the JSON-backed keyword retrieval system but do not automatically transfer to a different retrieval backend (e.g., Qdrant with dense vector search), where ranking and score distributions may differ enough to shift the learned policies' effective reward distribution. Re-generating the simulation cache after a backend change and retraining is the minimum validation step; this is tracked as GAP-5 in Section 10.

Single-labeler ground truth (new threat, called out explicitly). All 86 routing labels, all 86 "correct photo" labels, and the grade cutoffs used to score the eval were produced by the system designer. No second labeler reviewed the ground truth; no inter-rater reliability (e.g., Cohen's κ) was computed. Two distinct sources of error are therefore invisible in the reported metrics: (a) routing labels that I judged canonical but that a second labeler would reasonably disagree with (the ∼10 queries flagged in the Routing Accuracy Regression Analysis are the tip of this iceberg), and (b) systematic labeler preference—if I tend to label borderline behavioral-vs-factual queries as factual because that is how I think about my own photos, the system is implicitly tuned to my classification biases. The minimum fix is a second independent labeler on a sampled subset (e.g., 20 queries) with a Cohen's κ report; the stronger fix is multi-label routing ground truth. I did neither in this project, and the headline metrics should be read accordingly.

14.2 External Validity

Held-out set is not external validation—read before citing generalization numbers. The 22-query held-out split is drawn from the same user, the same 53-photo library, and the same labeler as the 64-query training split. It tests whether the RL policies overfit to the specific training queries; it does **not** test whether the policies generalize to a different user, a different photo distribution, or an independently-produced ground truth. Any reader citing "held-out 77.3% retrieval"

as evidence of external generalizability is over-reading the number. The true external-validity claim this report supports is: "the learned policies do not degrade on unseen queries drawn from the same distribution."

Single-user knowledge base. Results are from one user's 53 iPhone photos. The RL architecture is domain-agnostic, but learned parameters would require retraining on different photo distributions. The offline simulator makes retraining cost-free; what it does not make cost-free is producing an independent ground-truth set for the new user, which is the actual bottleneck for cross-user validation.

LLM dependency. Base retrieval quality depends on GPT-4o Vision's OCR and entity extraction. The offline simulator decouples RL training from the LLM.

14.3 Construct Validity

Retrieval accuracy definition. Binary metric (expected photo appears anywhere in results) prioritizes recall; rank-aware metrics (MRR, NDCG) would capture ranking quality.

Confidence grade discretization. The DQN's 5-action space maps to letter grades. The requery action mitigates over-commitment by gathering additional evidence.

14.4 Statistical Conclusion Validity

Seed-based CIs. With 5 seeds, 95% CIs use the t-distribution (4 df). We report exact p-values, Cohen's d, and significance stars.

Ceiling effects. Several metrics are near 100%, compressing the space for RL improvement. Silent failure rate—the safety-critical metric—shows the clearest benefit (9.4% $\rightarrow$ 1.3% for Full RL on the 64-query training split; 4.7% $\rightarrow$ 2.8% on the full 86-query set with the Recommended config).

15 Ethical Considerations

Privacy. PhotoMind operates on personal data (receipts, food photos, behavioral patterns). All training data stays local; no query patterns are transmitted externally. The RL system learns from query-outcome patterns that may encode purchasing habits, dietary choices, and location information.

Silent failure prevention. The DQN's reward matrix penalizes silent failures most severely (-1.0). This prioritizes user safety over coverage—it is better to decline an answerable query than to confidently return wrong information about personal financial data.

Routing bias. The bandit's reward signal derives from ground-truth labels assigned by the photo owner. Learned policies reflect the labeler's intent, not objective truth. Different users' labeling conventions would produce different routing policies.

Explainability. The DQN logs the full state vector and selected action for each query, enabling post-hoc analysis. Future work should surface per-component attribution directly in user-facing responses.

Consent and transparency. Any deployment should inform users that the system improves through query feedback. The current implementation stores no user-identifiable data beyond the local knowledge base.

16 Future Work

16.1 Immediate Extensions

- **Vector database migration:** Replace JSON knowledge base with ChromaDB/Qdrant for $O(\log n)$ retrieval at 10K+ photos
- **Incremental ingestion:** Watch `photos/` with `watchdog` for real-time processing
- **Structured output enforcement:** Pydantic response models in agent tasks
- **Multi-modal re-ranking:** Pass top-K candidates to GPT-4o Vision for fine-grained re-ranking

16.2 Production Deployment Roadmap

The phases below are ordered by **dependency and risk**: each phase removes a blocking constraint that the next phase assumes is resolved. Phase 1 addresses data integrity (without ACID guarantees, concurrent access under scale would corrupt the knowledge base). Phase 2 addresses retrieval latency (without a vector backend, the system cannot serve 1K+ photo corpora in interactive time). Phase 3 assumes reliable, fast retrieval exists before exposing trust indicators to users. Phase 4 requires all prior infrastructure before multi-user isolation is meaningful.

1. **Phase 1 (Reliability) [Must]:** SQLite/PostgreSQL backend for ACID transactions and concurrent access safety, structured output enforcement via Pydantic `response_model` to eliminate citation-omission failures, query feedback loop recording every query-outcome pair for RL retraining
2. **Phase 2 (Scale) [Should]:** Vector database migration (ChromaDB/Qdrant) for $O(\log n)$ retrieval at 1K+ photos, batch-parallel ingestion with rate limiting, DQN inference caching for repeated query patterns
3. **Phase 3 (UX) [Should]:** User-visible confidence trust indicators ("I'm fairly confident" vs. "I found something but I'm not sure"), explicit feedback mechanism closing the online learning loop, lightweight performance analytics dashboard
4. **Phase 4 (Multi-user) [Could]:** Per-user RL policy instances trained on individual photo distributions, federated policy initialization from aggregate data, privacy-preserving on-device training with no query data leaving the user's machine

MoSCoW prioritization summary. The features already implemented (GPT-4o Vision analysis, 3-strategy retrieval, confidence grading, graceful decline, contextual bandit routing, DQN confidence calibration, offline RL simulator) are all **Must** or **Should** priorities that were built under the project's time constraints. Embedding-based semantic search and the Qdrant vector backend are **Could** priorities—the Qdrant backend is implemented as opt-in (`REPOSITORY_BACKEND=qdrant`) but JSON remains default for the current 53-photo scale. Online RL training, multi-user federated learning, and multilingual support are **Won't (this version)** items deliberately deferred as out of scope for the single-user prototype.

16.3 Research Directions

- Learned requery policy (choosing which strategy to try next, not random)
- Noise injection robustness experiments across $\sigma \in \{0.01, 0.05, 0.1\}$
- Rank-aware retrieval metrics (MRR, NDCG) for finer-grained evaluation
- Online learning from user feedback to continuously adapt RL policies

17 Reproducibility

All experiments can be reproduced from a clean checkout:

Listing 3: Complete reproduction pipeline.

```
# 1. Environment setup
~/.pyenv/versions/3.10.14/bin/python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. Run RL training (7 configs x 5 seeds)
python -m src.rl.training_pipeline

# 3. Run evaluation suite (86 queries, generates ablation tables)
python -m eval.run_rl_evaluation

# 4. Generate all 10 visualization figures
python -m viz.generate_figures

# 5. Run unit tests (45 tests across 7 test classes)
pytest tests/ -v
```

Seeds: $\{42, 123, 456, 789, 1024\}$, set via `torch.manual_seed()`, `numpy.random.seed()`, `random.seed()`.

Hardware: Apple M-series MacBook (ARM64, CPU-only PyTorch). Full training completes in under 60 seconds.

API cost for reproduction: $0. All RL training and evaluation runs entirely offline.

18 Conclusion

PhotoMind demonstrates a complete, production-motivated agentic system with four specialized agents, two distinct workflows, five tools (three built-in + two custom), an 86-query evaluation harness, and an RL extension that reduces silent failures—**evaluated on a single user's 53-photo iPhone library with single-labeler ground truth**. Every headline metric that follows is scoped to that regime.

The system integrates two RL approaches—contextual bandits (Thompson Sampling) for query routing across four search strategies and a multi-step DQN for confidence calibration—into a CrewAI multi-agent architecture. The DQN's non-terminal requery action enables multi-step episodes with structurally relevant discounting. Both components train offline via deterministic simulation at zero API cost.

A 7-configuration ablation study across 5 random seeds yields three principal findings:

1. The DQN is the primary contributor to silent failure reduction and decline accuracy improvement.
2. Thompson Sampling complements the DQN on ambiguous queries, while UCB's exploration variance compounds with DQN error modes—a novel interaction-effect analysis for modular RL.

3. The **Recommended (Rule+DQN)** configuration offers the best risk-adjusted performance: 75.6% routing accuracy (identical to baseline), 85.7% decline accuracy, and 2.8% silent failures (from 4.7% baseline) on the full 86-query evaluation.

Summary results:

Config	Retrieval	Routing	Silent Fail	Decline
Recommended (Rule+DQN)	86.0%	75.6%	2.8%	85.7%
Full RL (Thompson+DQN)	86.0%	71.6%	4.0%	77.1%
Held-out (22 queries)†	77.3%	63.6%	8.2%	68.0%
Base system (20 queries)	80%	75%	5%	100%

†The held-out split is drawn from the same user, photo library, and labeler as the training set. It is a within-corpus overfitting sanity check, not an independent external validation. Metrics on this split should not be compared directly to the training-set configurations above; they reflect distributional shift within a single-labeler regime. See Section 14 for the full validity analysis.

The reduction of silent failures is the most important result. For a system operating on personal financial and behavioral data, minimizing confident-but-wrong answers is more valuable than marginal accuracy improvements.

Corpus boundary (the single most important caveat on every number above). All results are bounded by a single-user, 53-photo corpus where the builder produced both the photo metadata and the test queries. The 22-query held-out set is a sanity check, not external validation—it is drawn from the same user and photo distribution as the training queries. Generalization to different users, photo volumes beyond 1,000, or unseen photo categories would require retraining and re-evaluating on a corpus whose ground-truth labels were produced independently. The headline metrics should be read as "what this system can achieve in this regime," not "what this system will achieve in production."

Latency reconciliation (the second caveat every reader should see). The end-to-end mean query latency on the CrewAI hierarchical pipeline is ∼45 seconds (§13, Challenge 12), driven by three sequential GPT-4o round-trips per query (Controller → Retriever → Synthesizer). This is a genuine weakness of the full-agent path. It is mitigated—not hidden—by three production fast-paths documented in §13 and §12: (1) the `query -direct` CLI path calls `PhotoKnowledgeBaseTool._run` with RL routing but skips CrewAI orchestration, taking < 1 s at zero API cost; (2) the `POST /api/query/stream` endpoint exposes the full pipeline over Server-Sent Events so the user sees token-level progress rather than a 45-second wall; (3) the `LRUQueryCache` (maxsize 128, TTL 300 s) in `api/server.py` makes repeated queries sub-millisecond. The honest production story is: the full-agent path is the research configuration that drives the ablation numbers; the fast-path is what a user would actually interact with. Both paths share the same RL-routed retrieval and the same DQN confidence grading, so the safety guarantees transfer.

AI Tools Acknowledgment

This project used AI-assisted development tools during both implementation and documentation. In the interest of academic transparency, their roles are described below.

Code development. Cortex Code (Snowflake's Claude-backed coding agent) and Claude were used as pair-programming assistants for portions of the implementation—including scaffolding modules, debugging, and writing unit tests. All architectural decisions, algorithm designs, reward matrix values, and evaluation methodology were authored by the student. AI-generated code was reviewed, tested, and integrated manually.

Documentation editing. This report was polished using an AI critique-and-revise loop. A Claude-based reviewer agent was used to flag gaps in authorial voice, stale metrics, and missing caveats. The student reviewed each flagged item, decided which to accept, and rewrote the affected sections. The AI did not generate the technical content, experimental results, or analytical conclusions—it served as a structured reviewer.

React frontend. The web dashboard (`web/`) was scaffolded with AI assistance and added in a single commit. The student designed the component structure and API integration; the AI generated boilerplate React/MUI/TypeScript code.

What was NOT AI-assisted. The following were produced entirely by the student: domain choice, system architecture, CrewAI agent prompt text, search strategy algorithms, RL formulation (bandit + DQN decomposition, reward matrix, offline simulator design), all 86 hand-labeled test cases, the Project Retrospective (19 issue logs), and the experimental analysis including the "bandit is diagnostic, not operational" finding.

Caveat audit trail — distinguishing AI-flagged from human-originated items. To clarify the provenance of self-critical content in this report: the corpus boundary caveats (Section 1, Section 18), the single-labeler threat (Section 14), and the binary metric limitation (Section 8) were all identified by the student during system testing—they reflect limitations the builder observed firsthand while evaluating their own system. The AI reviewer subsequently flagged presentation gaps (e.g., the metric caveat was buried in a footnote rather than front-loaded, the latency reconciliation lacked the three-fast-path enumeration) and suggested structural improvements to make these caveats more visible to readers. In every case, the student decided whether to accept the structural suggestion and rewrote the prose. No caveat in this report was invented by the AI—the AI's role was to surface presentation deficiencies in limitations the student had already identified. The 19 issue logs in the Project Retrospective were authored before any AI review pass and reflect the student's independent engineering judgment.

Repository: https://github.com/raghuneu/PhotoMind
Implementation: CrewAI · GPT-4o Vision · PyTorch · Python 3.10.14 · React · FastAPI
RL Training: Thompson Sampling (contextual bandit) + Multi-Step DQN (confidence with requery)